

17.02.2024г.

ООП
Лекция 2

побитови операции
функции от по-висок ред } припомняне

namespaces - пространство от имена

```
namespace A {  
    void f() { ... }  
    namespace B {  
        void f() { ... }  
    }  
}
```

A::B::f()

```
namespace A {  
    void f() { ... }  
    namespace B { f() { ... } }  
    using namespace B;  
    void g() { f(); }  
}
```

// ще се компилира
// няма да се компилира

enum :



enum



unscoped

типично
кастване;

енумераторите
са в scope на
енума

/ enum class



scoped

X


```
enum A{
```

```
  x - глобално  
};
```

```
int main()
```

```
{ int x=7; - в scope-a на main-a
```

```
  "shadow"
```

```
  x++;
```

```
}
```

```
A::x
```

// не може да го променим, защото е константа

```
enum A{ над този enum class B{
```

```
  x  
};  
}
```

скрива

този

```
int main()
```

```
{ int x;
```

```
}
```

компилира се

```
enum Test {
```

```
  a=0,
```

```
  b=12
```

```
}
```

$\text{sizeof}(\text{Test}) = \text{sizeof}(\text{int}) = 4$ // 6 ранките на нашия курс приемат, че е колкото размера на int

```
enum Test2 {
```

```
  a=0,
```

```
  b=UINT_MAX+1
```

```
}
```

```
sizeof(Test2) = 8
```

// прави по-голям размер

// може да е грешно

```
sizeof(Test2) = 4
```


enum Test3: char { ^{→ правим енул от тип char (приемаме, те променливите ще са char)}

a=8,
X- b=80000 } // не е валидна стойност за char
// зависи от компилатора дали ще се компилира
sizeof(Test3)=1 // ако е само "a=8"
// char има 1 байт

Структури

• деклариране A obj;
A* ptr = new A(...)

• достъп до елементите
obj.x++; // икрементиране
↓ оператор

(*ptr).x++;
ptr->x++; } еквивалентни са

• масиви от инстанции
A arr[10];
A* ptr = new A[n];
...

delete[] ptr;

• влагане на инстанции

• подраване на инстанции във функции

f1(A obj) // по копие

f2(const A obj) // константно копие

f3(A& ref) // променяме ранните

f4(const A& ref) // constant reference

f5(A* ptr) // по указател

f6(const A* ptr) // константен указател

можем да избякаме функциите:

1 2 3 4 5 6

1 2 ~~3~~ 4 ~~5~~ 6

1 2 3 4 5 6

1 2 ~~3~~ 4 ~~5~~ 6

1 2 3 4 5 6

1 2 ~~3~~ 4 ~~5~~ 6

f1(A obj)

f1, f3, f5 правят
копие

Връщане на референции
на инстанции

от функции
A f()
{ A obj;
 obj.data....
 return obj;
} // връщане копие

↑ указател

(A*) f() {
 A obj;
 return &obj;
}

} // коректен код

const A & f() {

(A obj);
 return obj;

} // връщане референция,
която е const и нито един
инстанцията се удебелява

f3(A & obj)
{ f5(&obj);
}

f6(const A* ptr)
{ f4(*ptr);
}

връщане референция

A & f() {
 A obj; → умира в края
 obj.data... на scope
 return obj;
}

} // коректен код

A* f() {
 A* ptr = new A {...};
 return ptr;
}

} // ще работи, т.е. валидно
// не знаем дали паметта ще се
изтрие.

← { Валидно е, но
Няма да го правим

Размер на инстанции

struct A {

char ch; // 1 големина

int a; // 4 големина

}

⚠ Всеки тип има Alignment req.
def: Разликата между всеки два
последователни адреса, но,
които можем да разположим
дадена променлива.

• int е 4 байта

можем да поставим int на адрес 0, 4, 8, 12, 16, 20, ...

↑
потенциални адреси

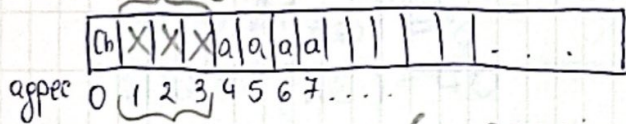
$\text{alignof}(\text{int}) = 4$

Примитивен тип:

T

$\text{sizeof}(T) == \text{alignof}(T)$

padding



те нямат никаква роля

struct A {

char ch; // 1

int a; // 4

}

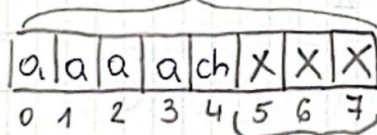
Struct A1 {

int a;

char ch;

};

обект A1



обект A1

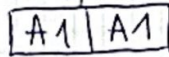
$\text{sizeof}(A1) = 8$

padding

Получаваме padding,

защото ако направим масив

A1 arr[2];



int може да варира

int32_t

→ знаков тип с 32 бита / 4 байта

uint32_t

→ без знак

// броят на битовете, за да получим байтове
режим на 8


```
struct A1 {
    int a;
    char ch;
};
```

$\text{alignof}(A1) = 4$

Примерни правила: (приемете ги по уговорка)
 ⚠ За да правим масиви, големината на структурата или обекта трябва да се дели на размера на най-големия примитивен елемент.

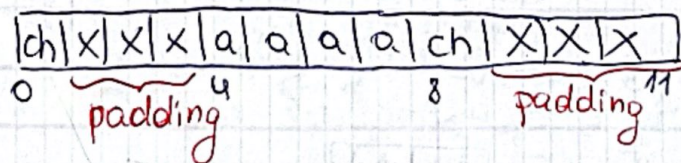
Всяка примитивна елементарна трябва да е на адрес, кратен на големината ѝ.

на колко MB компилираме
 $\text{int}^* \text{ptr}$ $\times 64 \rightarrow 8 \text{ байта}$
 $\text{sizeof}(\text{ptr})$ $\times 32 \rightarrow 4 \text{ байта}$

за зорачи теория

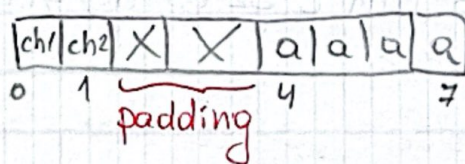
```
struct A3 {
    char ch;
    uint32_t a;
    char ch;
};
```

$\text{sizeof} : 12$
 $\text{alignof} : 4$



```
struct A4 {
    char ch1;
    char ch2;
    uint32_t a;
};
```

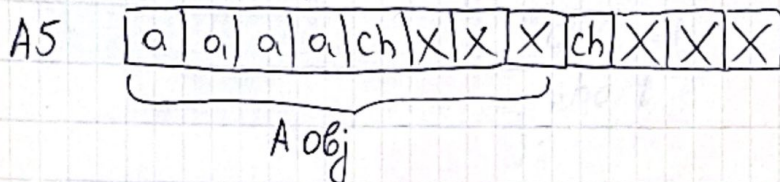
$\text{sizeof} : 8$




```
struct A5 {
    A obj;
    char ch;
};
```

```
A {
    uint32_t x;
    char ch;
};
```

$\text{alignof}(A) = 4$



// понеже align на A=4
то $\text{sizeof}(A5)$ трябва
да се дели на 4.

$\text{sizeof}(A5) = 12;$
 $\text{alignof}(A5) = 4;$

```
struct A6 {
```

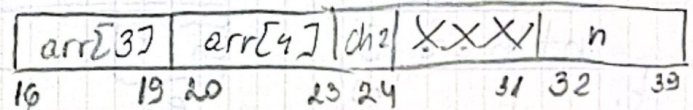
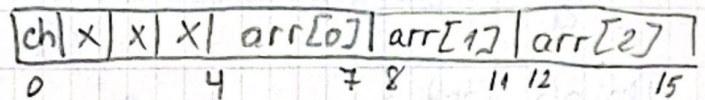
```
    char ch;
```

```
    uint32_t arr[5] // все едно е пет променливи по 4
```

```
    char ch2;    int64_t n;
```

```
};
```

$\text{alignof}(A6) = 8$
 $\text{sizeof}(A6) = 40$



```
struct A7 {
```

```
    char ch;
```

```
    int arr[5];
```

```
};
```

→ само ако е последен елемент,
моще да не поравняваме размер

когато място е останало

$\text{sizeof}(A7) = 4;$

```
struct Test {
```

```
    int_32_t a;
```

```
    char ch;
```

```
    char arr[3];
```

→ Все едно имам три клетки, които
са наближа

```
};
```



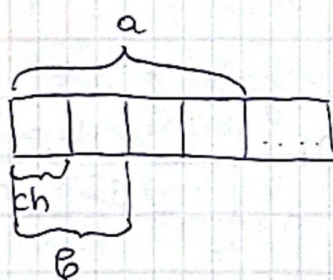
```
struct T {
    char ch;
    int a[];
    int b[];
};
```

// Няма да се компилира

Union S (обединение)

def: Последователност от полета, които заемат / споделят една и съща памет.

```
Union Test {
    int32_t a;
    char ch;
    int16_t b;
};
```



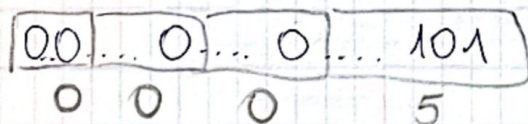
// започва от ляво
нагоре

def: Endianness

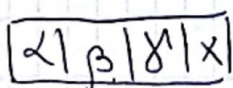
→ начин на подреждане на байтовете в една дума

int x | y | z | w

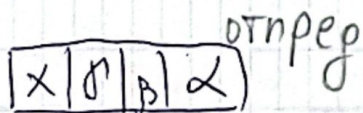
Байтът е число от 0 до ²⁵⁵~~256~~.



- Big endion



- Little endion // най-старшият отзв, най-младшият



Приемаме, че използваме Little endion.

```
int x = 5;
```

00	00	00	05
----	----	----	----

05	00	00	00
----	----	----	----

```
union T {
    int a;
    char ch;
    short b;
};
```

```
T object;
```

```
object.a = 75;
```

```
cout << object.ch
```

// принтираме
буквата със

ASCII символ = 75; // К

75	00	00	00
----	----	----	----

така приемаме

4 3 2 1 ← байтове

ch a b

```
cout << obj.b // 75
```

```
obj.a = 50000 ≥ 216 - 1
```

```
cout << obj.b // #50000
```

```
union Person {
    student s;
    Teacher t;
};
```

различна интерпретация (полиморфизъм)

Задача. Направете функция, която връща дали работим в Little endion. Да връща true или false.

```
union LittleEndianCheck
```

```
{ char ch;
```

```
  uint16_t n = 1;
```

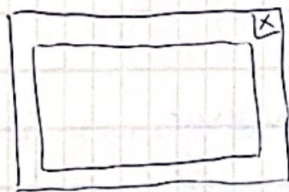
```
};
```

ch
01 00
n

⚠ Union са предназначени за използване на точно едно поле от тях.

Ако достигнем поне 2 полета, то това е Undefined Behavior.

Потоци и текстови файлове

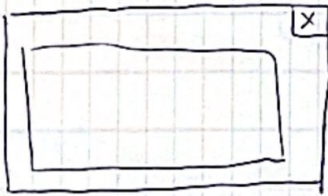


потоци за изход
>>>>

потоци за вход
>>>>

def: Потоци - последователност от байтове, "несогени" в определена посока.

Вход:

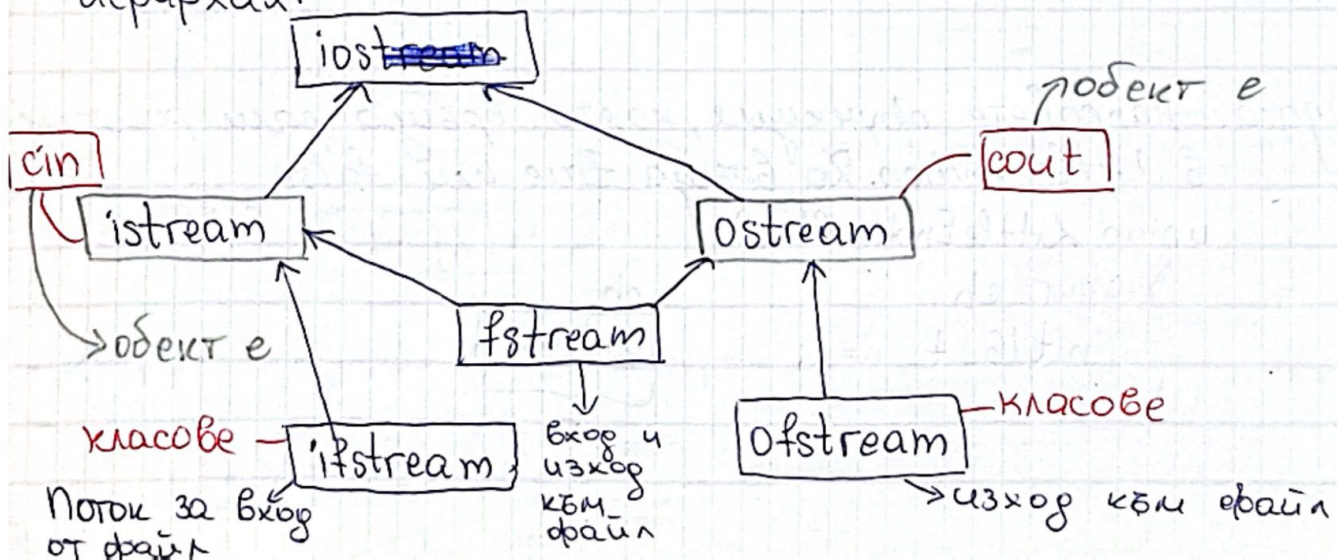


Изход:

- стандартен изход (конзола)
- файл
- друга програма

cout и cin са видове потоци

Иерархия:



~~Вход:~~
Форматиран:
`<<` 
`<<`
тип

Вход:
`int a;`
`>> a`

Неформатиран:

is `get()` → взема един символ
os `put(char ch)`

is `getline(буфер, размер)`

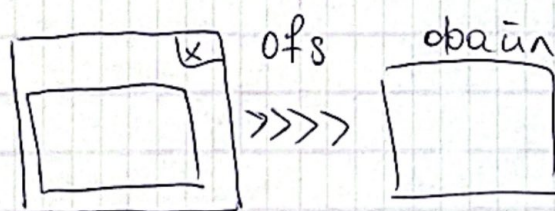
↓
земе до нов ред или до запълване на буфера

```
ofstream ofs("file.txt");
```

```
ofs << ... // работим с него както със съст
```

```
! if(!ofs.is_open()) { // проверка дали е отворен  
    // error handling;    файл
```

```
}  
ofs << 37;  
ofs.put('x');
```



```
int x;
```

```
cin >> x; "AB" // потокът се "сгупва"
```

Трябва да се съобразяваме какво има във файла

```
ifs.eof() // край на потока
```


Зарата | Функция, която приема име на файл и връща брой на
редовете във файла.

A | B | C

get указател